MINISTRY OF EDUCATION AND RESEARCH
OF THE REPUBLIC OF MOLDOVA
Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

Vasilean Vitalie
Group FAF-241

Report

Laboratory Work No. 2
Operating Systems
Topic: Meet the OS You Will Build (xv6)

Checked by:
univ. assist. / lect. Patricia Reițman
Department of Software Engineering and Automation
FCIM, UTM

Chisinau – 2026

Contents

1 INTRODUCTION AND GOAL **3**

1.1 Introduction 3

1.2 Lab Objective 3

2 PART 1: SETUP AND BUILD ENVIRONMENT **4**

2.1 Toolchain Verification and Compilation 4

2.2 Analysis and Observations 4

3 PART 2: USING XV6 AS THE UNIX IT IS **5**

3.1 Command Execution in xv6 5

3.2 Analysis and Observations 5

4 PART 3: READING THE SOURCE CODE **6**

4.1 Inspection of Userland Source Files 6

4.2 Analysis and Observations 7

5 CONCLUSIONS **8**

1 INTRODUCTION AND GOAL

1.1 Introduction

Modern monolithic production kernels such as Linux encompass millions of lines of code, complicating direct pedagogical study of kernel subsystems. In contrast, xv6 is a clean, modern re-implementation of Dennis Ritchie's and Ken Thompson's classic Unix Version 6 (V6), ported to the 64-bit RISC-V architecture. xv6 illustrates foundational concepts—virtual memory paging, process context switches, hardware interrupts, system call traps, and file system disk blocks—in a concise, readable code base.

1.2 Lab Objective

The primary objective of this laboratory work is to configure the xv6 development toolchain, boot the operating system under hardware emulation, interact with its userland environment, and study core system call interfaces at the source code level. The specific focus includes:

- establishing the cross-compilation toolchain (`riscv64-linux-gnu-gcc`) and booting xv6 in the QEMU emulator;
- interacting with standard xv6 utilities (`ls`, `cat`, `echo`, `grep`, `wc`) and evaluating command pipelines;
- analyzing utility source code implementations (`user/ls.c`, `user/cat.c`) and tracing user-to-kernel transitions via system call tables (`user/user.h`, `kernel/syscall.h`, `kernel/sysfile.c`).

2 PART 1: SETUP AND BUILD ENVIRONMENT

2.1 Toolchain Verification and Compilation

The RISC-V cross-compiler and QEMU system emulator were verified under Ubuntu WSL2. The xv6 repository was cloned, and the operating system was compiled using the native `Makefile`:

```
vitalie@Lenovo:~$ riscv64-linux-gnu-gcc --version | head -n 1
riscv64-linux-gnu-gcc (Ubuntu 13.3.0-6ubuntu2~24.04) 13.3.0

vitalie@Lenovo:~$ qemu-system-riscv64 --version | head -n 1
QEMU emulator version 8.2.2 (Debian 1:8.2.2+ds-0ubuntu1.2)

vitalie@Lenovo:~$ cd ~/xv6-riscv && make qemu
riscv64-linux-gnu-gcc -Wall -Werror -O -fno-omit-frame-pointer -ggdb -gdwarf-2 -march=rv64gc -
    MD -mcmodel=medany -ffreestanding -c -o kernel/entry.o kernel/entry.S
...
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 3 -nographic -
    global virtio-mmio.force-legacy=false -drive file=fs.img,if=none,format=raw,id=x0 -device
    virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0

xv6 kernel is booting

hart 2 starting
hart 1 starting
init: starting sh
$
```

Figure 2.1: Figure 2.1 – xv6 compilation and initial boot banner under QEMU RISC-V emulator

2.2 Analysis and Observations

a) **Cross-compilation necessity:** Because the host machine runs an x86_64 processor while xv6 targets the RISC-V (rv64gc) architecture, a cross-compiler (`riscv64-linux-gnu-gcc`) is mandatory to generate valid machine instructions.

b) **SMP and Multi-hart startup:** During boot, xv6 detects multiple RISC-V hardware threads (harts) and initializes them sequentially (`hart 1 starting`, `hart 2 starting`) before spawning the userland `init` process.

3 PART 2: USING XV6 AS THE UNIX IT IS

3.1 Command Execution in xv6

Standard utilities and anonymous pipes were executed at the xv6 interactive shell prompt:

```
$ ls
.              1 1 1024
..             1 1 1024
README         2 2 2441
cat            2 3 36728
echo           2 4 35592
forktest       2 5 18136
grep           2 6 39792
init           2 7 35832
kill           2 8 35584
ln             2 9 35432
ls             2 10 38048
mkdir          2 11 35640
rm             2 12 35632
sh             2 13 56840
stressfs       2 14 36608
usertests      2 15 192800
grind          2 16 50744
wc             2 17 37192
zombie         2 18 35056

$ echo hello xv6
hello xv6

$ wc README
94 656 2441 README

$ ls | grep c
cat
echo
forktest
stressfs
usertests
wc
```

Figure 3.1: Figure 2.2 – Directory listing, utility execution, and pipe chaining in xv6

3.2 Analysis and Observations

a) **Output formatting of `ls`:** Unlike GNU `ls`, the xv6 utility prints four columns: name, file type (1 for directory, 2 for regular file), inode number, and file size in bytes.

b) **Pipe operation:** The command `ls | grep c` illustrates Unix stream composition: the shell forks two processes, creates an anonymous pipe via the `pipe()` system call, redirects `stdout` of `ls` to the write end, redirects `stdin` of `grep` to the read end, and lets data stream concurrently.

4 PART 3: READING THE SOURCE CODE

4.1 Inspection of Userland Source Files

The source code of `user/cat.c` and system call bindings were analyzed:

```c
// user/cat.c
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"

char buf[512];

void
cat(int fd)
{
  int n;

  while((n = read(fd, buf, sizeof(buf))) > 0) {
    if (write(1, buf, n) != n) {
      fprintf(2, "cat: write error\n");
      exit(1);
    }
  }
  if(n < 0){
    fprintf(2, "cat: read error\n");
    exit(1);
  }
}

int
main(int argc, char *argv[])
{
  int fd, i;

  if(argc <= 1){
    cat(0);
    exit(0);
  }

  for(i = 1; i < argc; i++){
    if((fd = open(argv[i], 0)) < 0){
      fprintf(2, "cat: cannot open %s\n", argv[i]);
      exit(1);
    }
    cat(fd);
    close(fd);
  }
  exit(0);
}
```

Figure 4.1: Figure 2.3 – The complete implementation of user/cat.c in xv6

System call declarations and dispatcher numbers were inspected in `user/user.h` and `kernel/syscall.h`:

```c
// user/user.h (system call prototypes)
int read(int, void*, int);
int write(int, const void*, int);
int open(const char*, int);
```

```
5  int close(int);
6  int exit(int) __attribute__((noreturn));
```

Figure 4.2: Figure 2.4 – System call prototypes in user/user.h

```
1  // kernel/syscall.h (system call indices)
2  #define SYS_fork    1
3  #define SYS_exit    2
4  #define SYS_wait    3
5  #define SYS_pipe    4
6  #define SYS_read    5
7  #define SYS_kill    6
8  #define SYS_exec    7
9  #define SYS_fstat   8
10 #define SYS_chdir   9
11 #define SYS_dup    10
12 #define SYS_getpid 11
13 #define SYS_sbrk   12
14 #define SYS_pause  13
15 #define SYS_uptime 14
16 #define SYS_open   15
17 #define SYS_write  16
18 #define SYS_mknod  17
19 #define SYS_unlink 18
20 #define SYS_link   19
21 #define SYS_mkdir  20
22 #define SYS_close  21
```

Figure 4.3: Figure 2.5 – Numeric system call vector mapping in kernel/syscall.h

4.2 Analysis and Observations

a) **System calls utilized by `cat.c`:** The program relies on `open()`, `read()`, `write()`, `close()`, and `exit()`.

b) **Absence of dynamic memory allocation:** `cat.c` allocates zero heap memory via `malloc()` or `sbrk()`. It relies on a static fixed-size buffer (`char buf[512]`), ensuring minimal memory footprint and zero allocation failure modes.

c) **Mechanism of user-to-kernel transition:** When `read()` is invoked in userland, the auto-generated assembly stub in `user/usys.S` loads the constant `SYS_read` (value 5) into register `a7` and executes the RISC-V `ecall` instruction. The CPU transitions into supervisor mode, traps into `kernel/syscall.c`, which indexes the system call jump table to invoke `sys_read()` in `kernel/sysfile.c`.

5 CONCLUSIONS

In this laboratory work, the architecture of the xv6-riscv educational operating system was investigated through environment deployment, execution testing, and source code inspection.

Booting xv6 in QEMU confirmed how an operating system initializes multi-core processor contexts and establishes userland execution with a clean terminal prompt. Interacting with basic commands demonstrated that fundamental Unix abstractions—including standard streams, descriptor redirection, anonymous pipes, and hierarchical namespaces—operate identically in xv6 and production Linux despite vast differences in code size. Finally, inspecting `user/cat.c`, `user/user.h`, and `kernel/syscall.h` clarified the complete lifecycle of a system call trap from userland execution down to kernel service routines.